A Reference Architecture for a Quantum-Centric Supercomputer

Seetharami Seelam
Chief Architect of Classical System Co-Design for QCSC
Distinguished Engineer

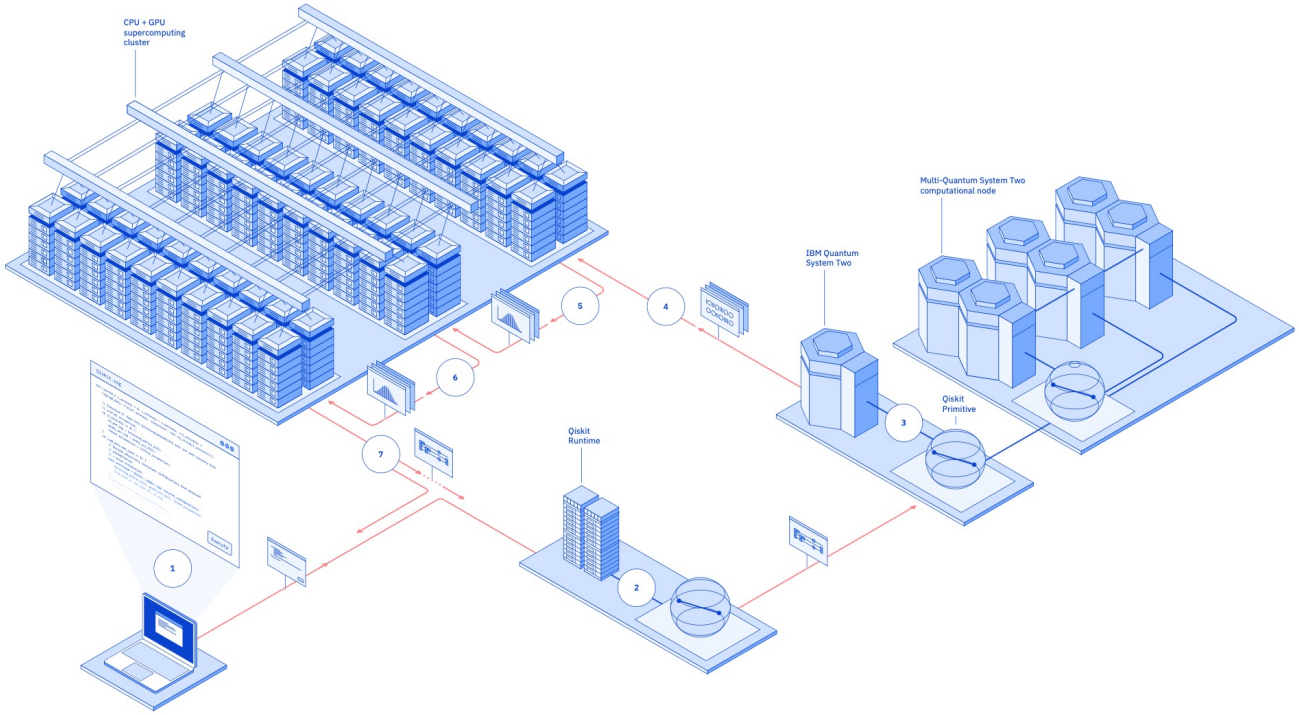
IBM Research

Paper: https://arxiv.org/abs/2603.10970

QPUs

+ CPUs

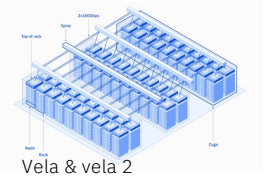
+ GPUs



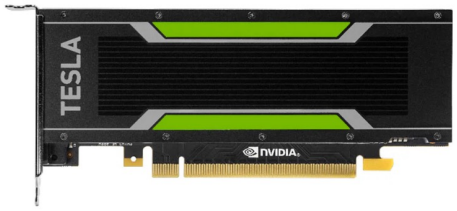
Quantum-centric supercomputing

for accelerated scientific discovery

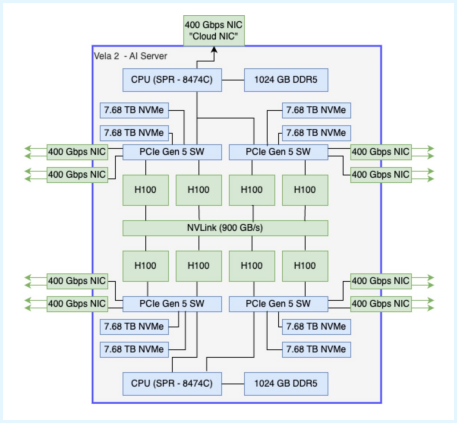
What is a GPU-centric supercomputer?



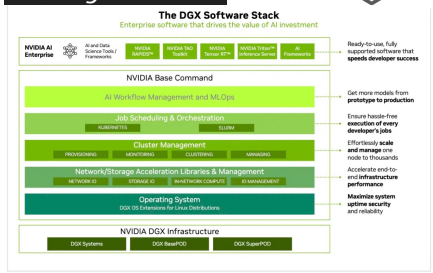
Vela & vela 2



PCI-E GPU



PyTorch vLLM llm-d



NVIDIA DGX H100 + Software stack

QPU

?

?

Workloads and their target training times: 2021

Workload requirements: circa. 2021

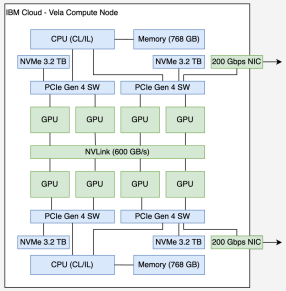
Model	Parameters	Input/Intermediate/output data	Target time with Vela
English RoBERTa	125M	51B words 1TB	<5 days (400 A100 GPUs)
XLM-RoBERTa	270M	334B words 3.2TB	<20 days (1440 A100 GPUs)
NMT model: 12+Lang	700M	2B words 312GB	<1 day (400 A100 GPUs)
NMT model: 50+Lang	~1B (12 layers)	40B words 4TB	<10 days (1250 A100 GPUs)

Workloads and their target training times: 2021

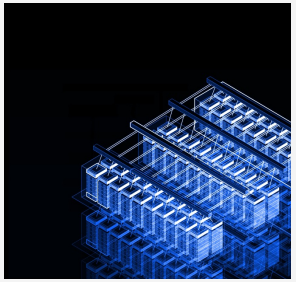
Workload requirements: circa. 2021

Model	Parameters	Input/Intermediate/output data	Target time with Vela
English RoBERTa	125M	51B words 1TB	<5 days (400 A100 GPUs)
XLM-RoBERTa	270M	334B words 3.2TB	<20 days (1440 A100 GPUs)
NMT model: 12+Lang	700M	2B words 312GB	<1 day (400 A100 GPUs)
NMT model: 50+Lang	~1B (12 layers)	40B words 4TB	<10 days (1250 A100 GPUs)

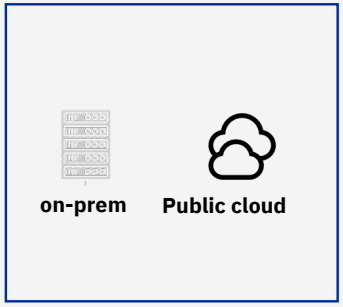
Provided directions for:



Technology choices
- CPUs, Memory, GPUs
- Local storage
- Network



System design
- Number of nodes
- Network (IB vs Ethernet)
- Network architecture
- Nodes per rack & Power



System Environment
- Cloud vs on-prem
- Multi-tenancy (customers)
- Access to other services
- Worldwide access

Platform design
- Workload mgmt. (Kueue)
- Network (Multi-nic CNI)
- Health checks (Autopilot)
- Topology scheduler (Sakara)

Workloads and their target training times: 2021 vs 2025

Workload requirements: circa. 2021

Model	Parameters	Input/Intermediate/output data	Target time with Vela
English RoBERTa	125M	51B words⍰1TB	<5 days (400 A100 GPUs)
XLM-RoBERTa	270M	334B words ⍰3.2TB	<20 days (1440 A100 GPUs)
NMT model: 12+Lang	700M	2B words⍰312GB	<1 day (400 A100 GPUs)
NMT model: 50+Lang	~1B (12 layers)	40B words⍰4TB	<10 days (1250 A100 GPUs)

Workload requirements: 2024-2025

Model	Parameters	Input/Intermediate/output data	Target time with Vela/Vela2
Granite-20B-code	20B	2.1T tokens 21TB	46 days (768 A100 GPUs)
Granite-8B	8B	4T tokens 50TB	31 days (1024 A100 GPUs)
Llama-ibm-70B	70B	4T tokens	41 days+ (768 H100 GPUs)

Lessons:

1. Requirements grew by O(~10^2-10^5) from 2021 to 2025 but **2021 models give us a direction**
2. None of the 2021 models are relevant anymore and 2024 models are quickly being replaced with newer MOE models
3. Start with just a few use cases, they be inaccurate, not relevant in 2 years, or irrelevant, that is all fine

Outline

6 use cases for Quantum + HPC

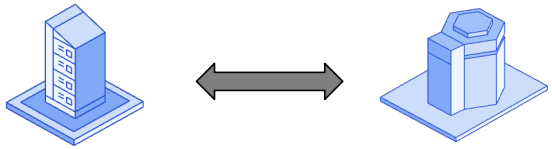
---

1 Reference Architecture

---

3 Phased deployment

Terminology



- Real-time: ~1-5us latency
- Near-time: ~10us - 1sec
- Batch-time: seconds to hours

- Temporal coupling:
  - Degree of time-dependent coordination
  - Strong or tight coupling: Coordination in real-time and near-time windows
  - Weak or loose coupling: Batch-time execution
- Spatial coupling:
  - Degree of physical proximity
  - Strong or tight spatial coupling: Co-location
  - Weak or loose coupling: Geographically distributed resource

Outline

5 use cases for Quantum + HPC

1 Reference Architecture

3 Phased deployment

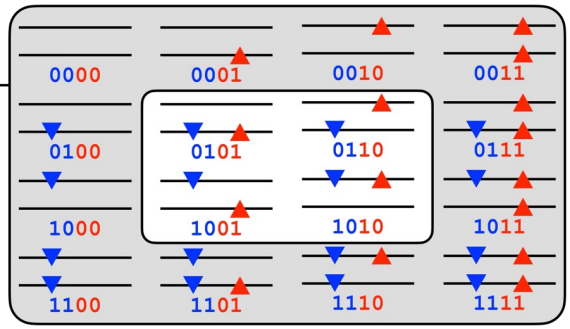
QCSC Workflow Example: Finding the ground state of a molecule

Exact Classical Solutions

$$\hat{H}|\Psi\rangle = E|\Psi\rangle$$

Sector $\binom{M}{N_\uparrow}\binom{M}{N_\downarrow}$

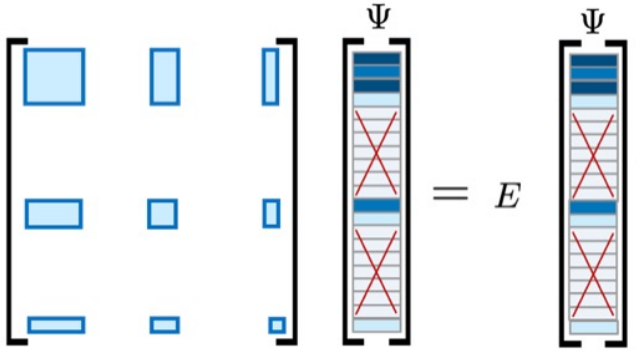
Fock space 2^{2M}

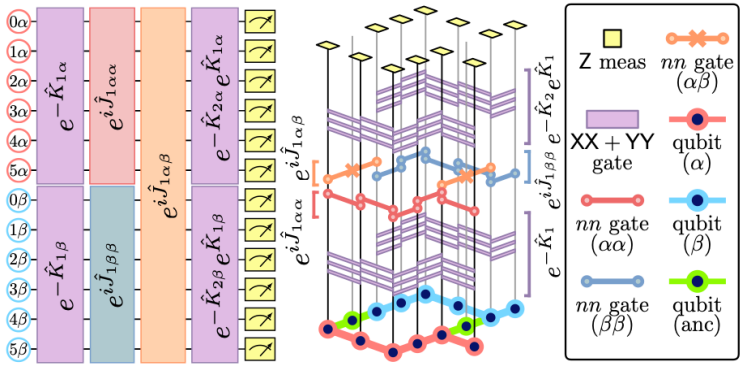


"Electron configurations" scale combinatorially with system size (number of electrons and orbitals). Quickly becomes intractable.

Approximate Methods

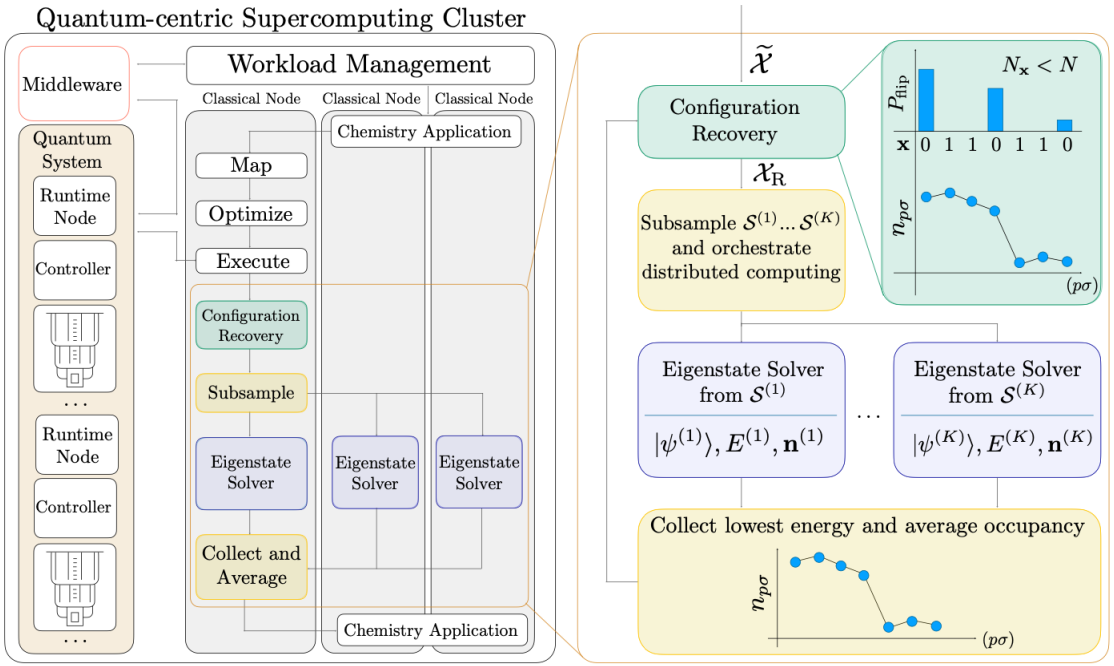
$$\sum_n H_{mn}\Psi_n = E\Psi_m$$

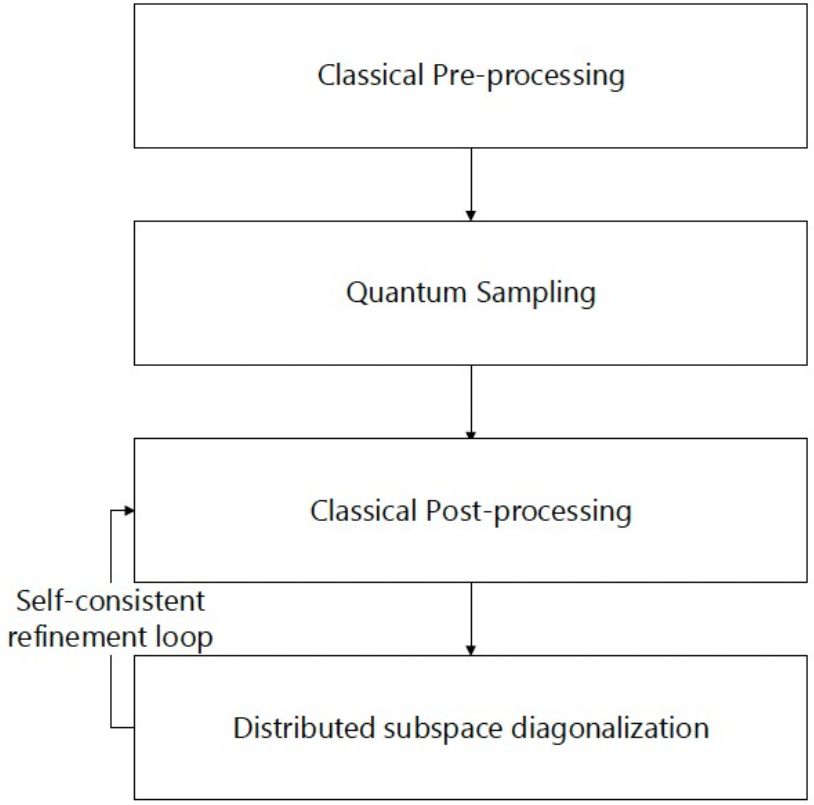




The ground state may be approximated by sparse linear combinations of electronic configurations. These can be efficiently sampled by quantum circuits.

QCSC Workflow Example: Finding the ground state of a molecule
Use case 1: Open-loop





Robledo-Moreno *et al. Science Advances* **11**, 25, (2025)

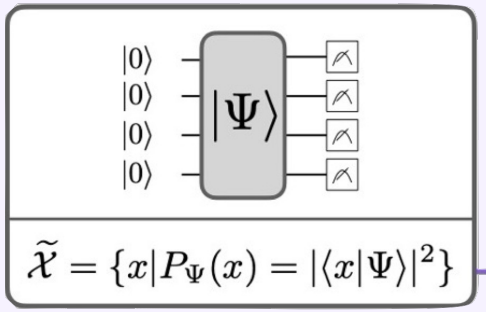
- Quantum and Classical parts of the algorithm are mainly sequential
- Required coupling of compute resources is loose
- This operates in a time frame we call ‘Batch time’ (seconds to hours latencies)

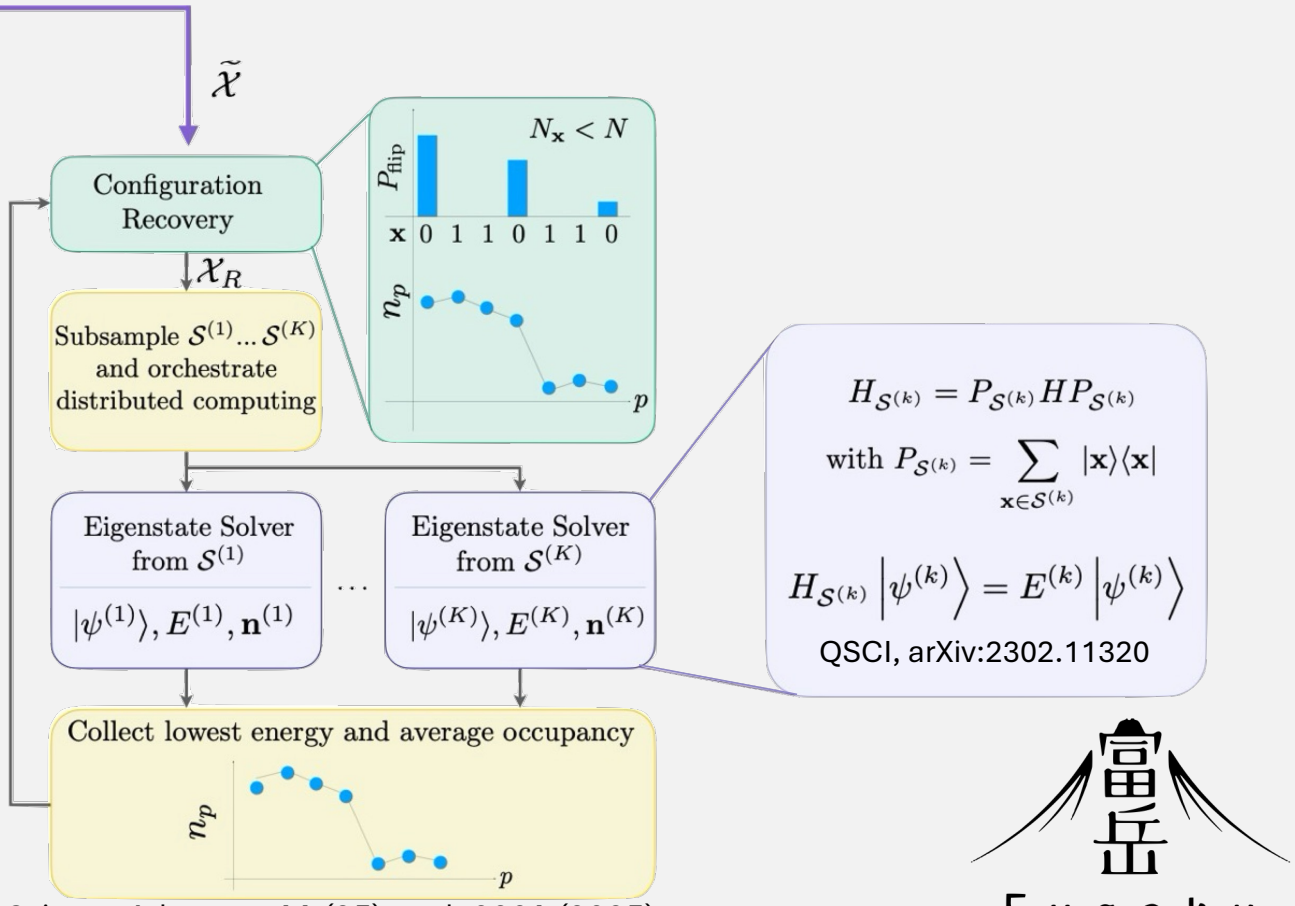
Key Observation 1: SQD exemplifies batch-time integration with loose coupling between quantum and classical HPC systems. These workflows do not require spatial or temporal co-location of systems, enabling flexible deployment across distributed or cloud connected infrastructures.

Sample-based quantum diagonalization workflow

Execute using Qiskit Runtime Primitives. Quantum compute generates samples from an exponentially large space

Sampler()





Science Advances 11 (25), eadu9991 (2025)

富岳
Fugaku

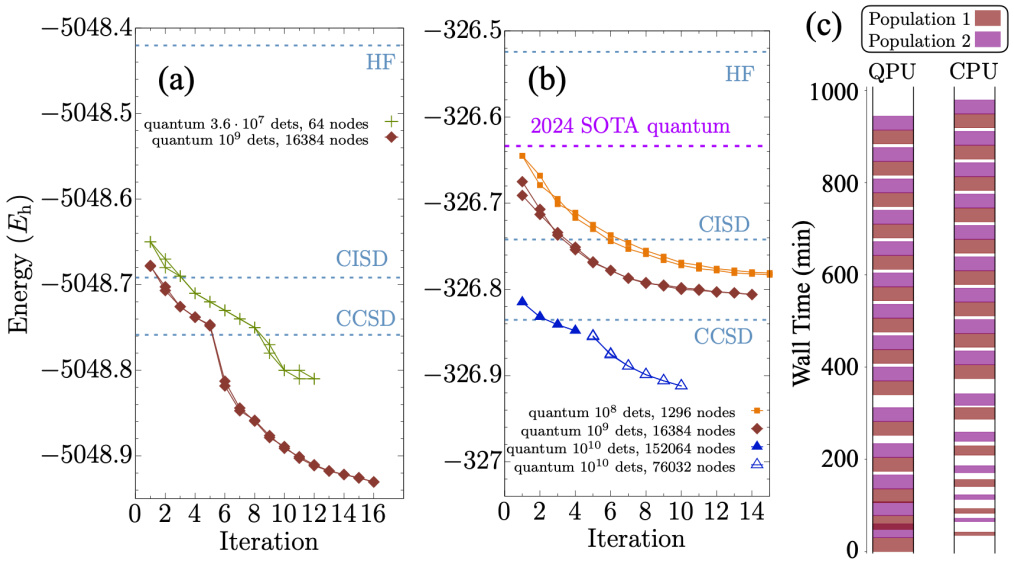
Quantum-centric supercomputing

77 qubits
10570 quantum gates
3590 two-qubit gates

富岳
Fugaku

~150,000 nodes @
32 GB
1024 GB/s
48 cores

Closed-loop calculations between a quantum processor and a classical supercomputer at full scale



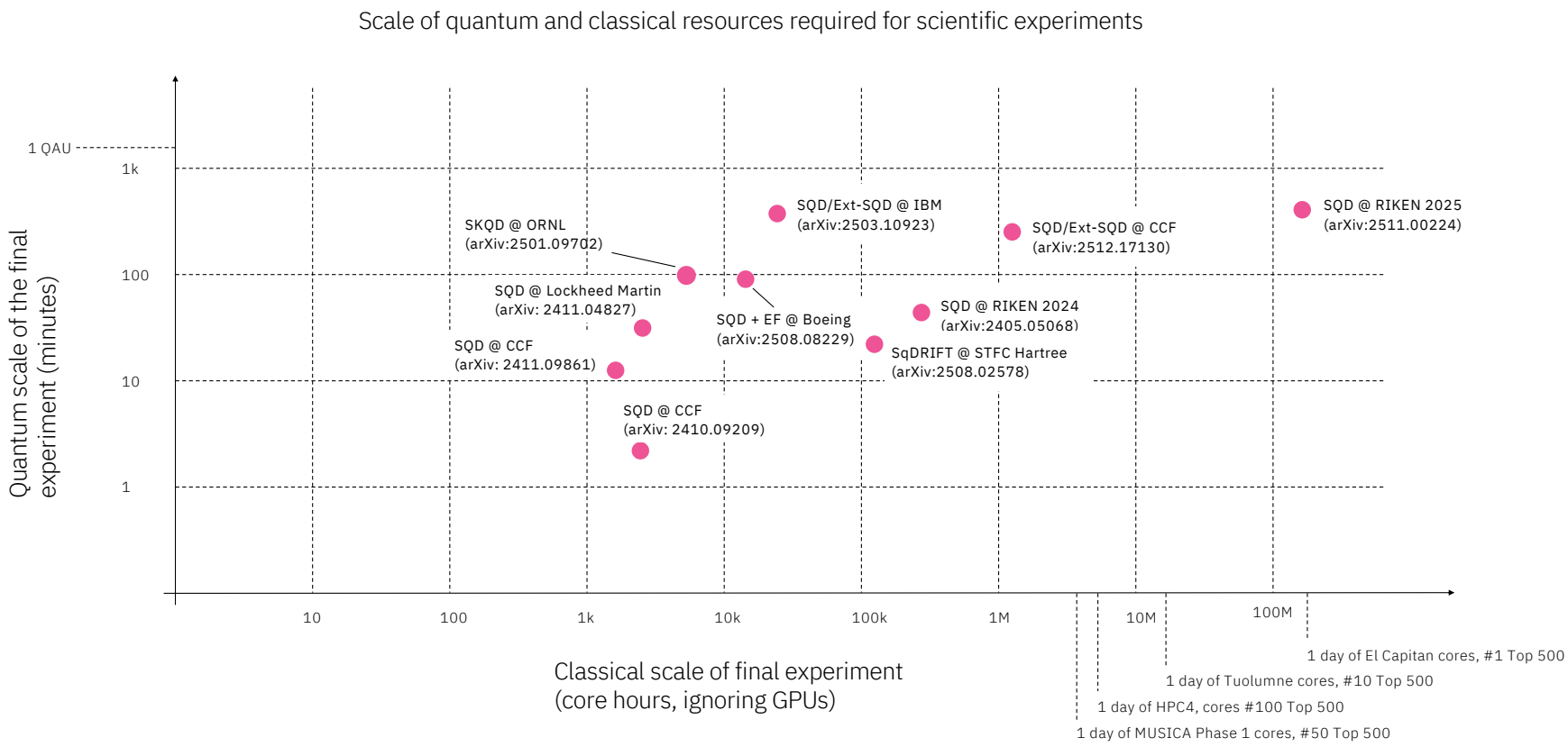
Subspace	(B,T,M)	Walkers	Total Nodes	Time [min]
10^7	(16,2,3)	4	384	$\sim$ 2
10^8	(36,6,6)	4	5,184	$\sim$ 5
10^9	(64,8,32)	4	65,536	$\sim$ 10
10^{10}	(576,12,22)	4	152,064	$\sim$ 15-20
10^{10}	(576,12,22)	2	76,032	$\sim$ 15-20

Quantum + 152,064 classical nodes

arXiv 2511.00224

IBM **Quantum** RIKEN

HPC and quantum workloads

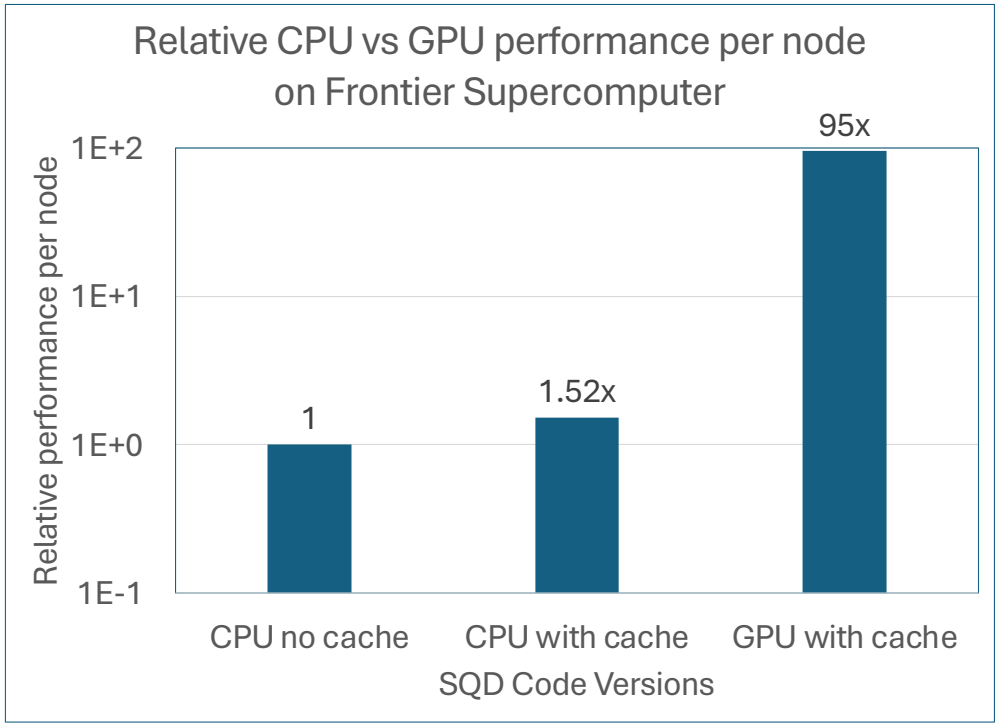


Diagonalization on GPUs

Implemented an offload strategy using **OpenMP offload**, achieving in the order of 100x-per-node performance boost on GPUs **without sacrificing accuracy**

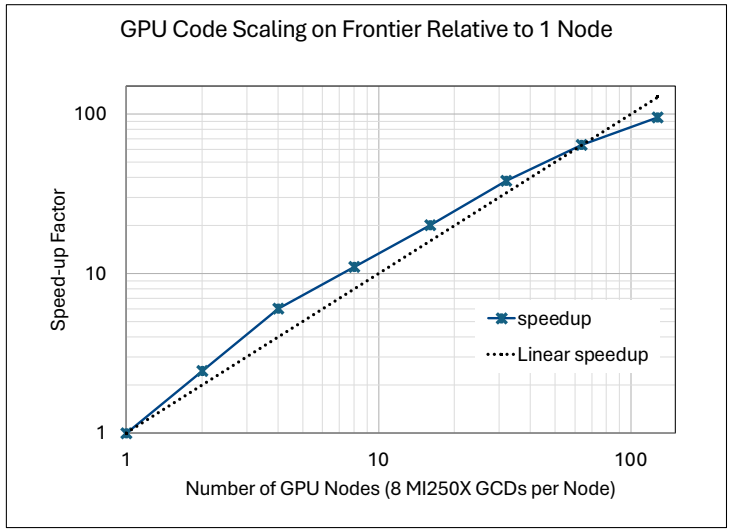
Key enhancements:

- √ Offloaded entire Hamiltonian matrix-vector multiply to GPU using OpenMP target directives
- √ Flattened 1D arrays for GPU compatibility and high memory access performance
- √ Avoided unnecessary CPU-GPU data movement with persistent GPU-resident data
- √ Pre-computed and cached configurations to avoid duplicate computations (at the cost of increased memory)

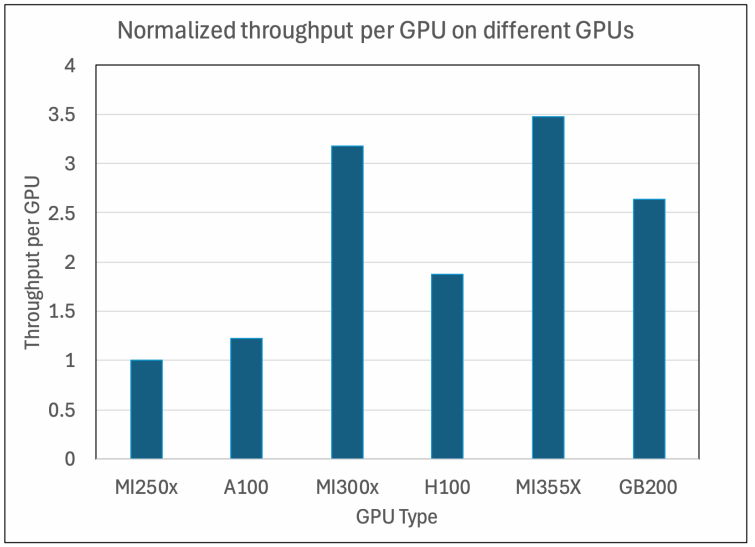


Scalable and portable diagonalization on GPUs

Diagonalization time lowered to manageable values for many applications of interest, using moderately sized and widely available resources



- Strong scaling performance on Frontier from 1 node to 128 nodes, 8 to 1024 GPUs
- Super-linear scaling up to 512 GPUs and 75% scaling efficiency at 1024 GPUs



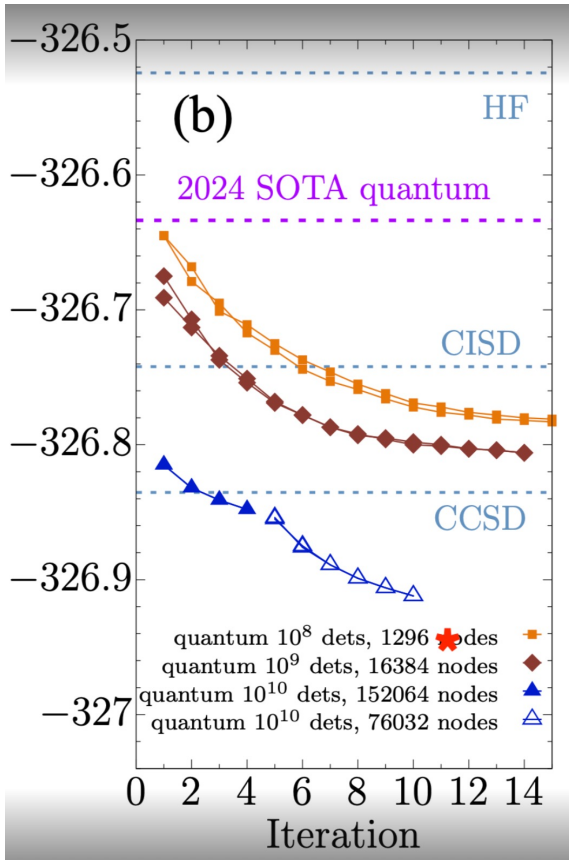
- Demonstrated portability on different GPUs: AMD MI300X, NVIDIA H100, and NVIDIA GB200
- MI300X achieves over 3x throughput improvement over MI250X

arXiv https://arxiv.org/abs/2601.16169

https://github.com/AMD-HPC/amd-sbd

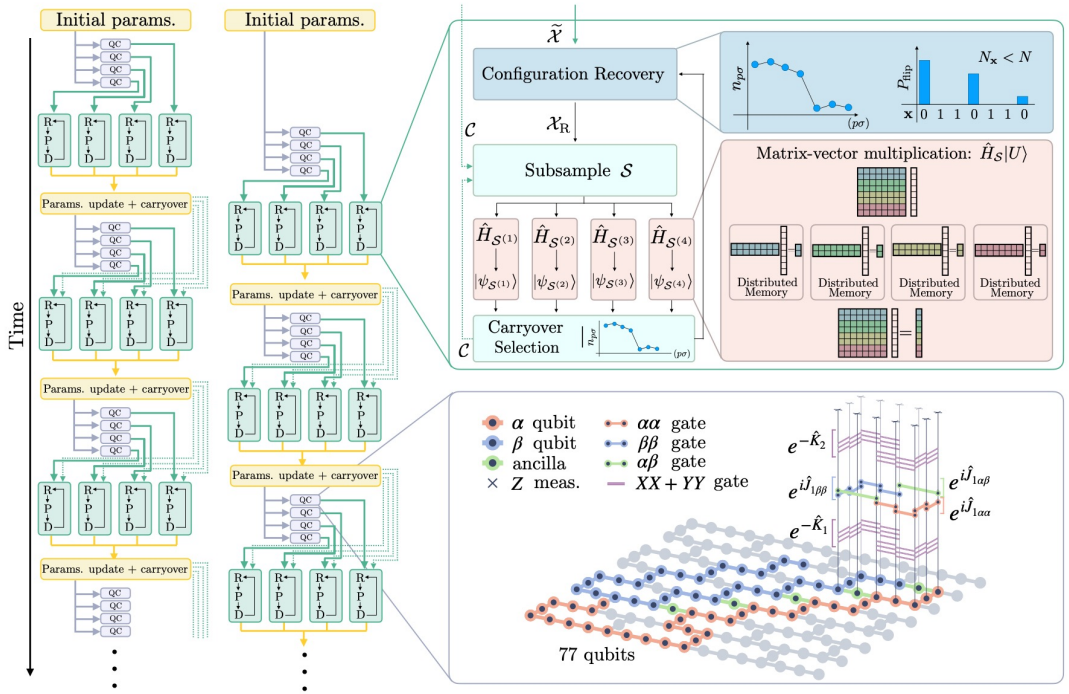
Is GPU the best device for this workload?

Can we do better with bigger subspaces?

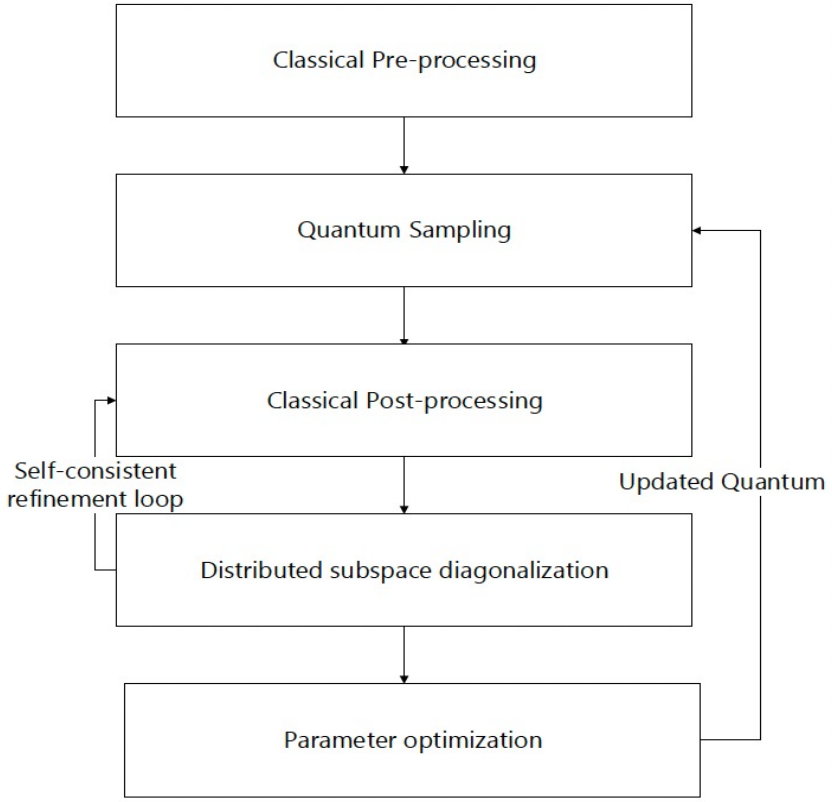


- ~29mEh lower than Riken experiment with 10x larger problem (10^11)
- ~**10x** improvement on x86 CPUs over ARM CPUs
- **150x** improvement with GPUs over CPUs
- Execution time: 784 H100 GPUs for 10.4 hours
- Explore GPU usage in Boeing use cases for SQD problems

QCSC Workflow Example: Finding the ground state of a molecule
Use case 2: Closed-loop



Shirakawa, *et al. arXiv*:2511.00224

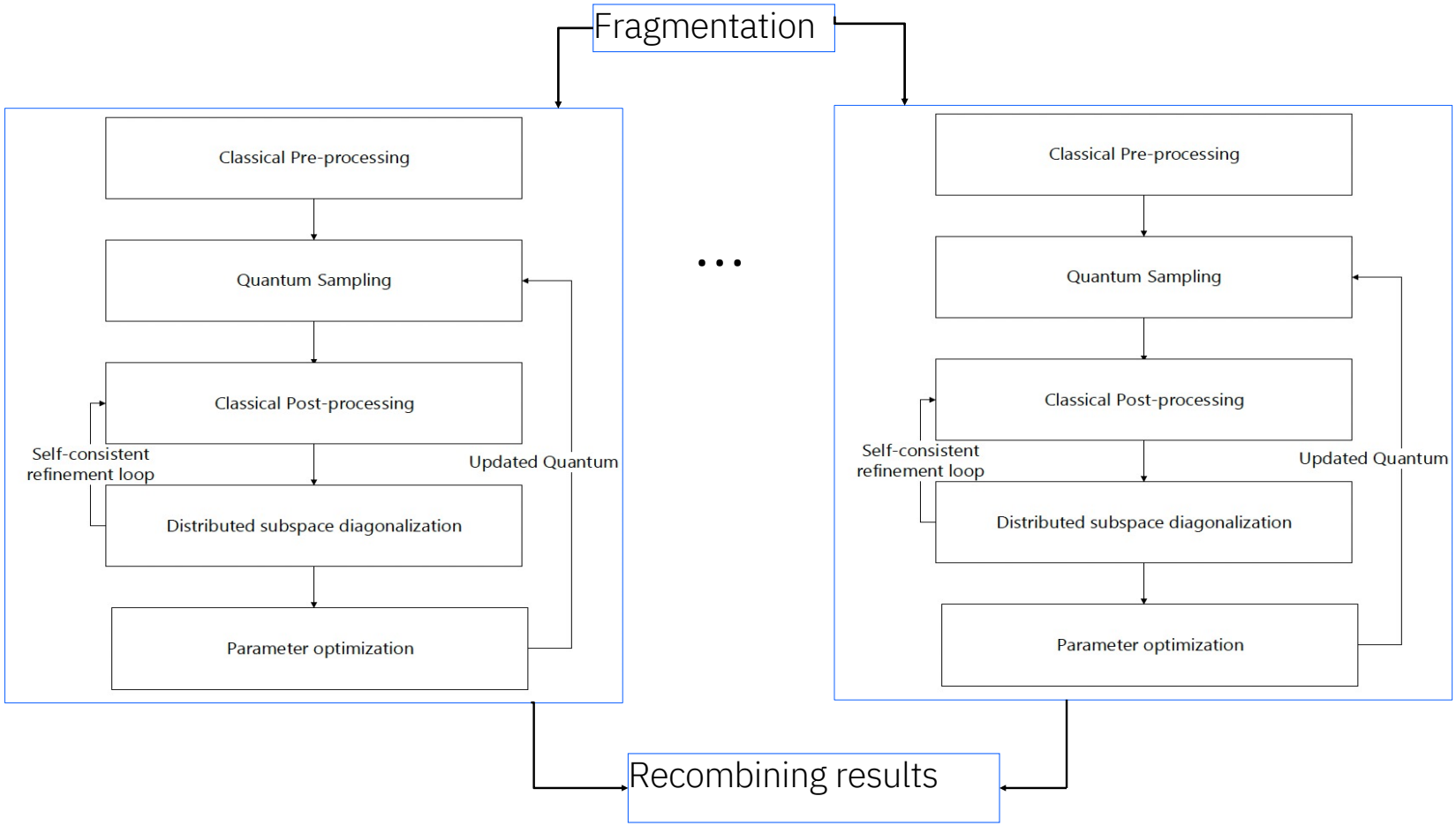


- Tighter temporal and spatial coupling: iteration output used to feed circuit parameters for next iteration
- Autonomy in system management and queue control is critical for workflow
- Involved latencies still operate at batch time

Key Observation 2: Closed-loop SQD exemplifies batch-time integration with temporal and spatial coupling. While the classical and quantum systems executed as distinct jobs, the output from one system (e.g., classical result) is used to refine the parameters for the next iteration on the other system (e.g., quantum circuit parameters), creating an iterative feedback loop. Spatial and temporal co-location of these systems enables efficient execution of such closed loop computations.

QCSC Workflow Example: Finding the ground state of a molecule
Use case 3: Horizontal scaling – 2 Supercomputers, 2 Quantum Computers

https://arxiv.org/abs/2605.01138



100's to 10's of thousands of fragments

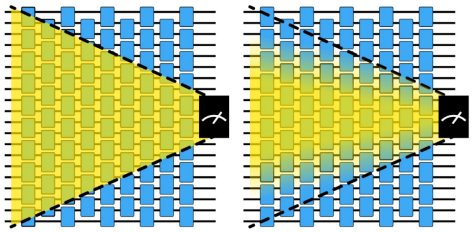
Summary of resource usage

	Open Loop	Closed loop	Parallel	Distributed
System size	1	1	~300	11,608–12,635
Number of QPUs	1	1	1	2
Number of Supercomputers	1	1	1	2
QPU time (minutes)	~15	210	600	6,360
CPU time (node-hours)	~16,000	~152,000	~100	~700,000
GPU time (GPU-hours)	0	0	0	~8,000
Number QPU-SC loops	1x14	2x14	300x14	12,000x14

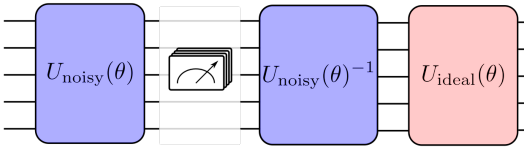
QCSC: Extending the computational reach of Quantum Computers with Quantum Error Mitigation
Use case 4: Embedding Classical Processing in the Quantum Workflow

EM methods that make use of classical HPC have emerged, that can reduce the sampling overhead of previous methods, in exchange for bias and classical runtime → larger circuit volumes at fixed error rates

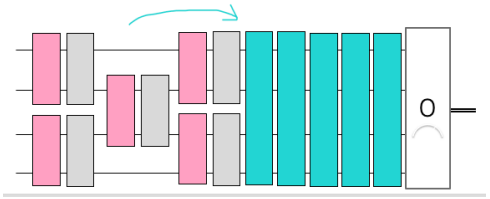
> Key Observation 3: Advanced error-mitigation techniques shift much of the computational burden to classical resources. Tensor-network contractions, Pauli propagation, and large sampling campaigns require substantial CPU/GPU compute, high-throughput data movement, and rapid orchestration between quantum execution and classical analysis. As quantum circuits scale, this access to HPC infrastructure becomes a key enabler for extending the reach of QPUs.



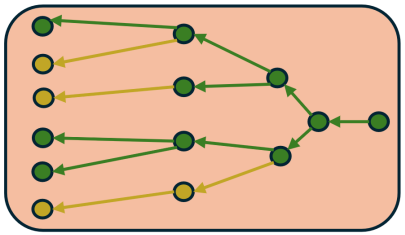
Shaded light cones
(arXiv:2409.04401)



Tensor network error mitigation
(arXiv:2307.11740)
algorithmiq

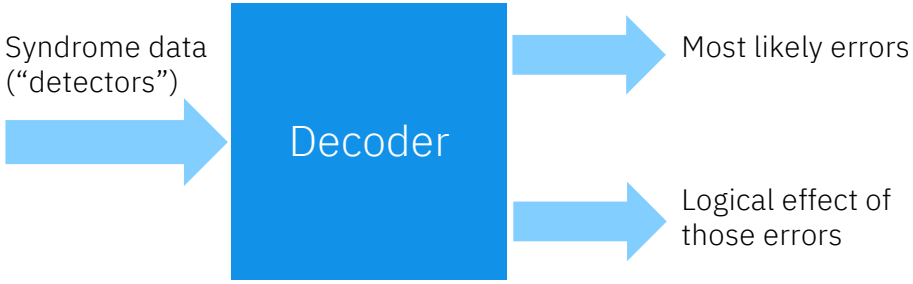


Propagated noise absorption



Quantum enhanced Pauli-path simulation

Real-time decoders for qLDPC codes



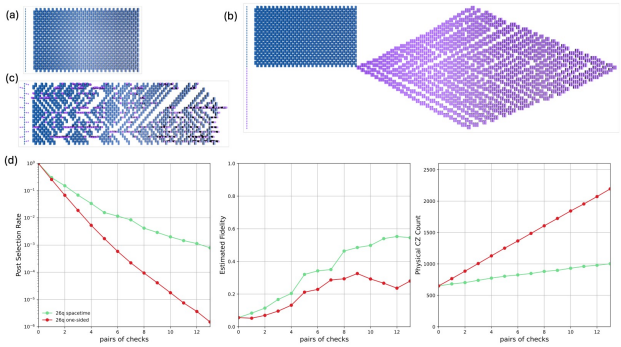
Critical metrics of a decoder include:

Throughput > 1 syndrome cycle / 1µs
Need to process syndromes fast enough to avoid accumulating a backlog

Latency < 10µs (logical instruction duration)
Logical measurements require decoder corrections. System stalls if applying a conditional operation.

Accuracy
Achieve the best possible logical error rate

QCSC: The case for Fault Tolerant Quantum Computing
Use case 5: Outer code research for fault-tolerant Quantum Error Correction



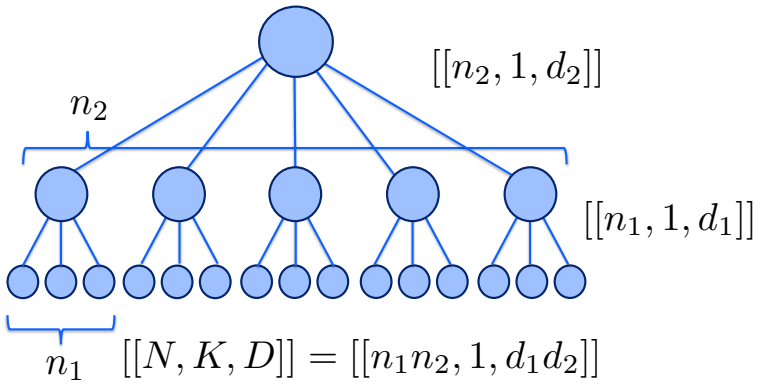
Comparing error detection in a Clifford circuit using spacetime checks vs. traditional stabilizer checks

Key Observation 4: Quantum error correction and detection research can be performed with flexible classical processors operating with quantum execution over high-bandwidth links.

QCSC: The case for Fault Tolerant Quantum Computing

Use case 6: Outer code research for fault-tolerant Quantum Error Correction

Hierarchical Quantum Error Correction



Concatenated code **corrects $d_1 d_2/2$ errors**

Hierarchy of gate durations:

Inner syndrome cycle: 1µs

Inner logical gate: 10µs

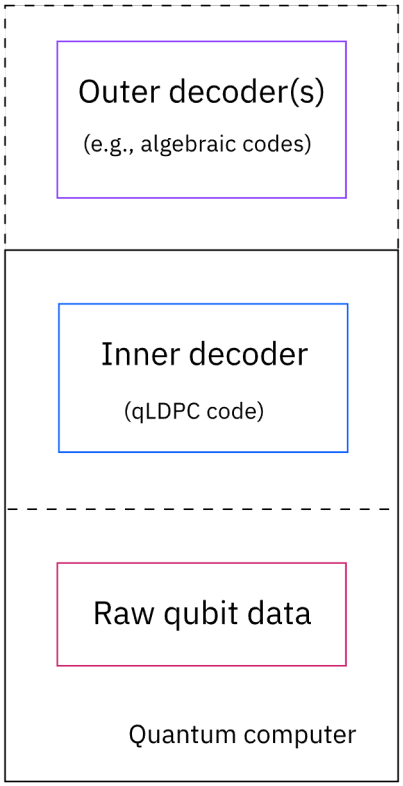
Outer syndrome cycle: >100µs

Inner hardware-adapted codes and outer algorithm-adapted codes:

Inner code has tight integration of QPU and control hardware (<10µs)

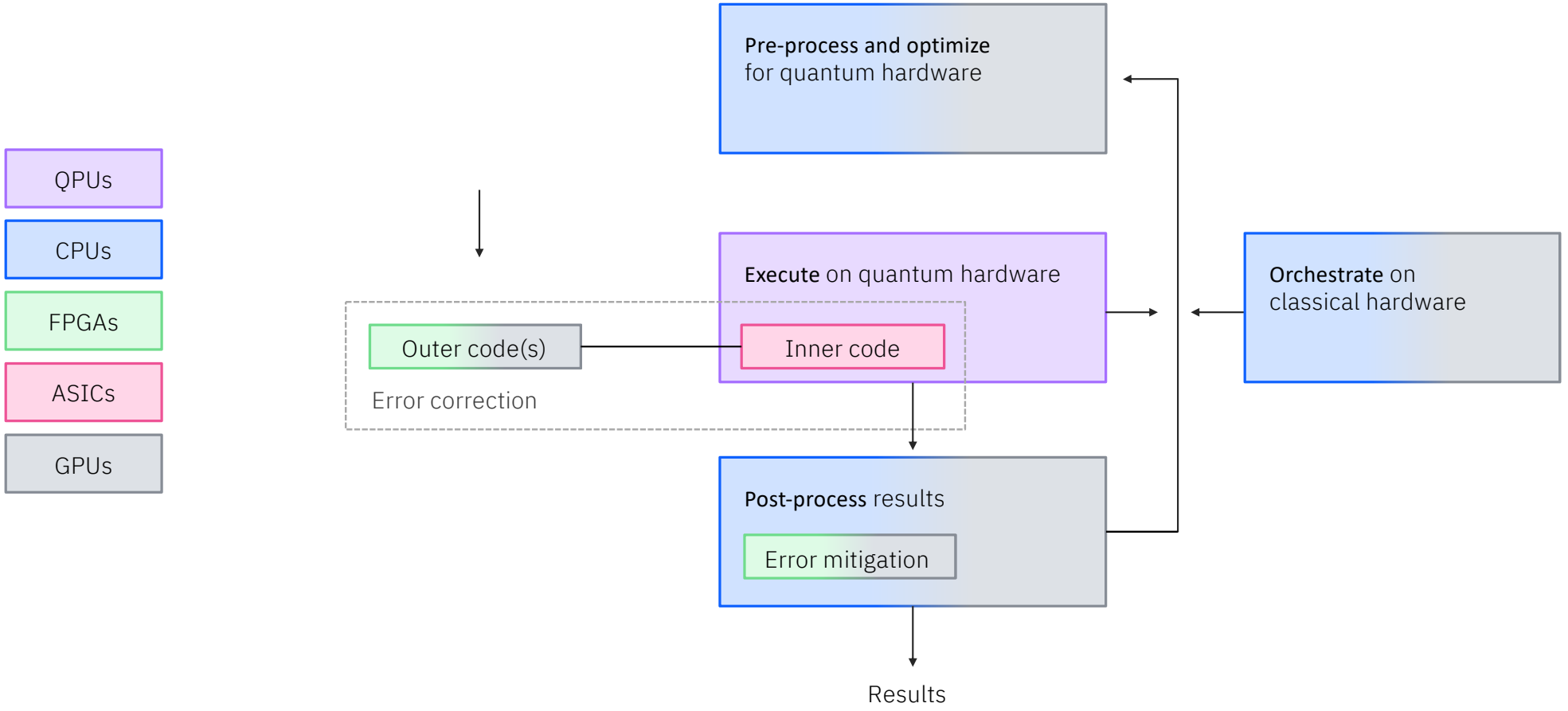
Outer codes produce syndrome measurements at a lower rate (approx. 100µs to 1ms) ideal for GPUs and advanced decoders

> Key Observation 5: Low latency (microseconds) scale-up classical systems of CPUs and GPUs have the potential to enable outer code research towards future fully-integrated fault-tolerant quantum error corrected systems, that inherently perform hardware-efficient real-time inner fault-tolerant error correction codes, such as qLPDC codes.



Hierarchical codes involve 2 levels of coding: (1) initial error correction of raw qubit data and (2) additional noise suppression

Quantum-classical workflow



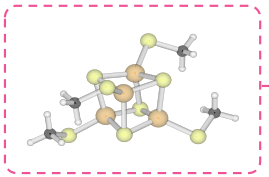
MAPPING OF USE CASES TO QPU–HPC INTEGRATION MODES

Use Case	Temporal Coupling	Spatial Coupling	Classical Resource	Key Requirement
1. Electronic Structure (SQD)	Batch-time	Loose	HPC Supercomputers	Co-location optional
2. Electronic Structure (Closed-loop SQD)	Batch-time	Tight	HPC Supercomputers	Co-location preferred
3. Large number of Atoms	Batch-time	Tight	HPC/GPU Supercomputers	Co-scheduling required
4. Error Mitigation	Batch-time / Near-time (future)	Tight	CPU and GPU HPC systems	High bandwidth
5. Open Loop QEC R&D	Batch-time	Loose	CPU, GPU, TPU HPC systems	High bandwidth
6. QEC outer decoder R&D	Near-time	Tight	CPU, GPU, TPU HPC systems	Low-latency

Quantum + HPC use cases

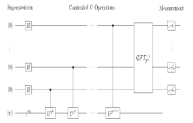
Time	Batch time (Seconds–hours)	Near time (10’s of microseconds to Milliseconds)	Real time (microseconds)
Use cases	Real applications Qiskit Functions Quantum physics **Quantum chemistry**	Map apps to quantum systems **Error mitigation** Circuit optimization Error correction research AI-based transpiler	**Fault-tolerant quantum system** Logical to physical qubit mapping Realtime error correction
Classical systems and scale	HPC systems Exaflop	Quantum + classical operating systems Hundreds of petaflops	Real-time operating systems ~1-100 teraflops
Infrastructure options	CPUs GPUs High-performance network Cloud On-prem	CPUs GPUs On-prem	GPUs for prototype Low-latency network FPGAs ASICs On-prem

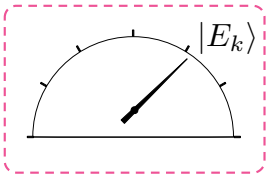
Hamiltonian simulation



[4Fe-4S] using an active space of 54 electrons in 36 orbitals from the TZP-DKH basis set

$e^{i(H\otimes\hat{p})}$





Pointer register that encodes energy eigenvalues

Measure energy E_k with probability $|\alpha_k|^2$

$$|\psi\rangle_{sys} = \sum_i \alpha |\psi_i\rangle_{sys}$$

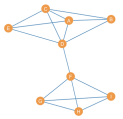
$$|0^n\rangle_{ptr}$$

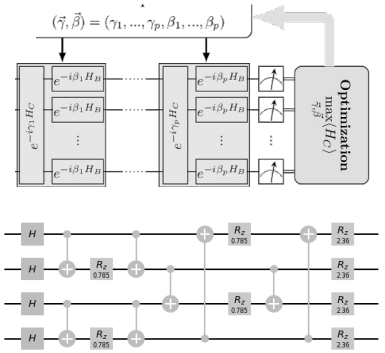
$$e^{i(H\otimes\hat{p})}|\psi\rangle_{sys}|0^n\rangle_{ptr} = \sum_{i=0}^{2^n-1} \alpha_i |\psi_i\rangle_{sys}|E_i\rangle_{ptr}$$

Optimization

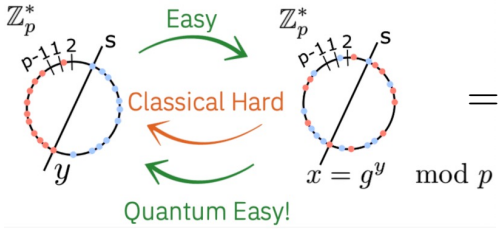
$$H_C = \sum_{ij} Q_{ij} Z_i Z_j + \sum_i b_i Z_i.$$

$$\min_{x\in\{0,1\}^n} x^T Q x$$





Quantum machine learning



$$= \frac{1}{\sqrt{|G|}} \sum_{g\in G} |f(g)\rangle|g\rangle$$

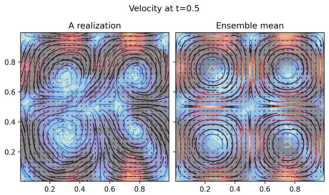
$$\sqrt{\frac{|B|}{|G|}} \sum_{g_0\in G/B} |f(g_0)\rangle \frac{1}{\sqrt{|B|}} \sum_{\mathbf{b}\in B} |g_0\mathbf{b}\rangle$$

$$f(g\mathbf{b}) = f(g) \qquad \forall\, \mathbf{b} \in B$$

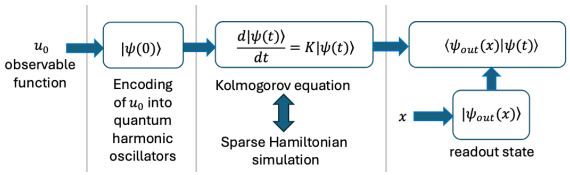
Differential equations

$$dX(t) = -\lambda X(t)dt + f(X(t))dt + \sqrt{q}\, dW$$

$$\frac{\partial}{\partial t}u(t,x) = \sum_{i=1}^{N}(-\lambda x_i + f_i(x))\frac{\partial}{\partial x_i}u(t,x) + \frac{q}{2}\Delta u(t,x)$$



Navier–Stokes



We need a reference architecture that supports all these workflows and evolves with them as systems and capabilities mature

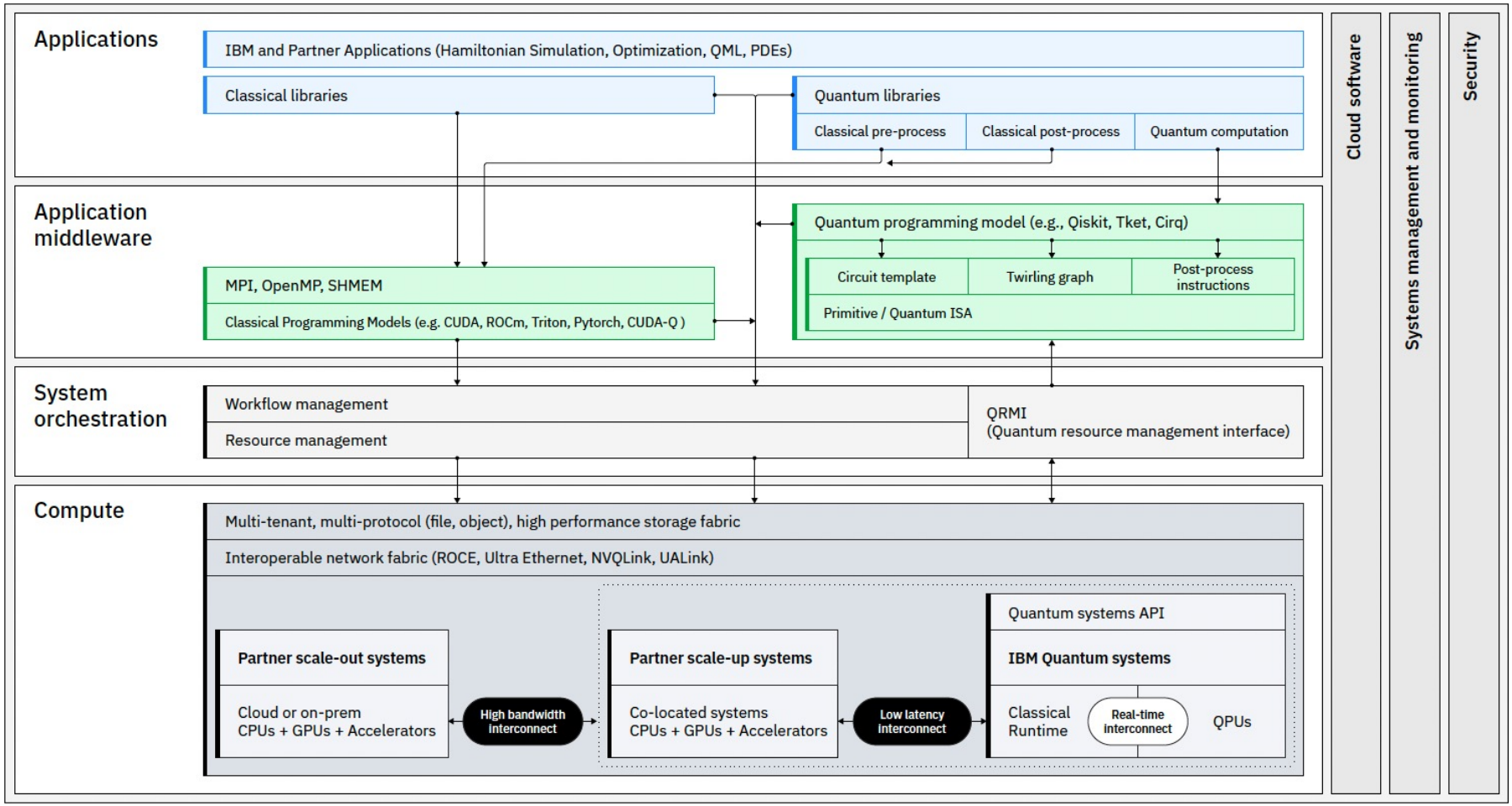
Outline

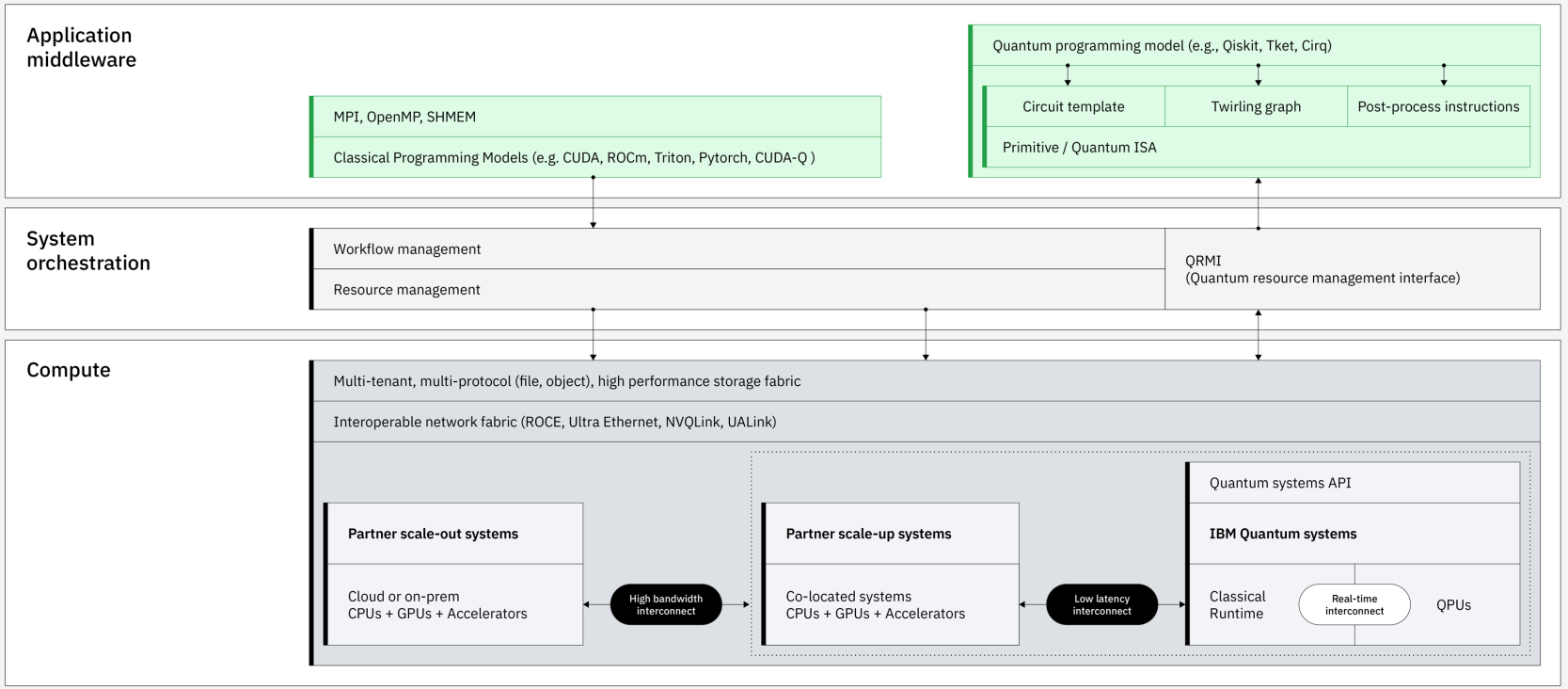
5 use cases for Quantum + HPC

1 Reference Architecture

3 Phased deployment

QCSC: Reference Architecture




Application middleware
MPI, OpenMP, SHMEM
Classical Programming Models (e.g. CUDA, ROCm, Triton, Pytorch, CUDA-Q)
Quantum programming model (e.g., Qiskit, Tket, Cirq)
Circuit template
Twirling graph
Post-process instructions
Primitive / Quantum ISA
System orchestration
Workflow management
Resource management
QRMI
(Quantum resource management interface)
Compute
Multi-tenant, multi-protocol (file, object), high performance storage fabric
Interoperable network fabric (ROCE, Ultra Ethernet, NVQLink, UALink)
Partner scale-out systems
Cloud or on-prem
CPUs + GPUs + Accelerators
High bandwidth interconnect
Partner scale-up systems
Co-located systems
CPUs + GPUs + Accelerators
Low latency interconnect
Quantum systems API
IBM Quantum systems
Classical Runtime
Real-time interconnect
QPUs

Quantum + Classical orchestration

Challenges:

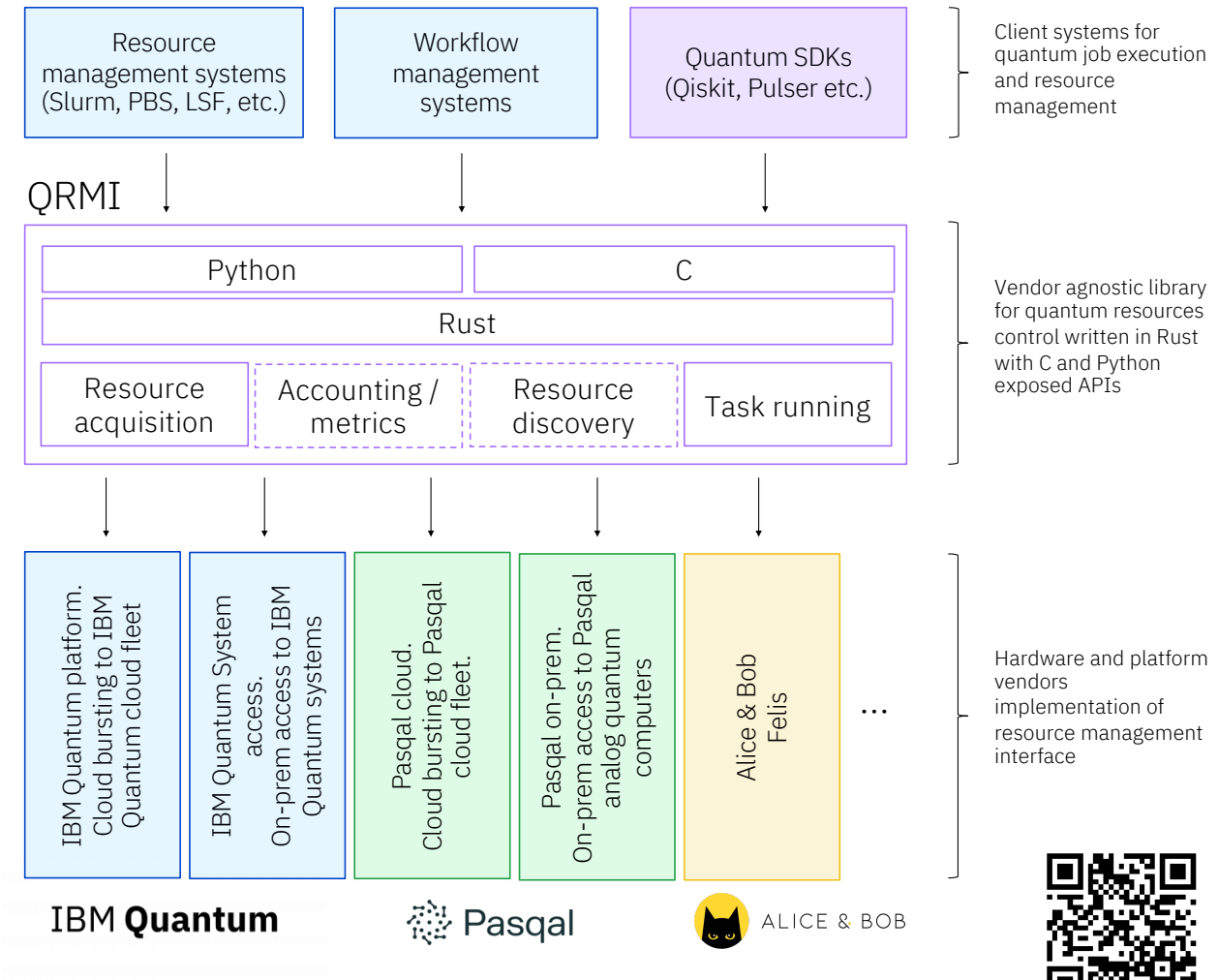
- Designing programs so that Quantum and Classical achieve comparable execution times

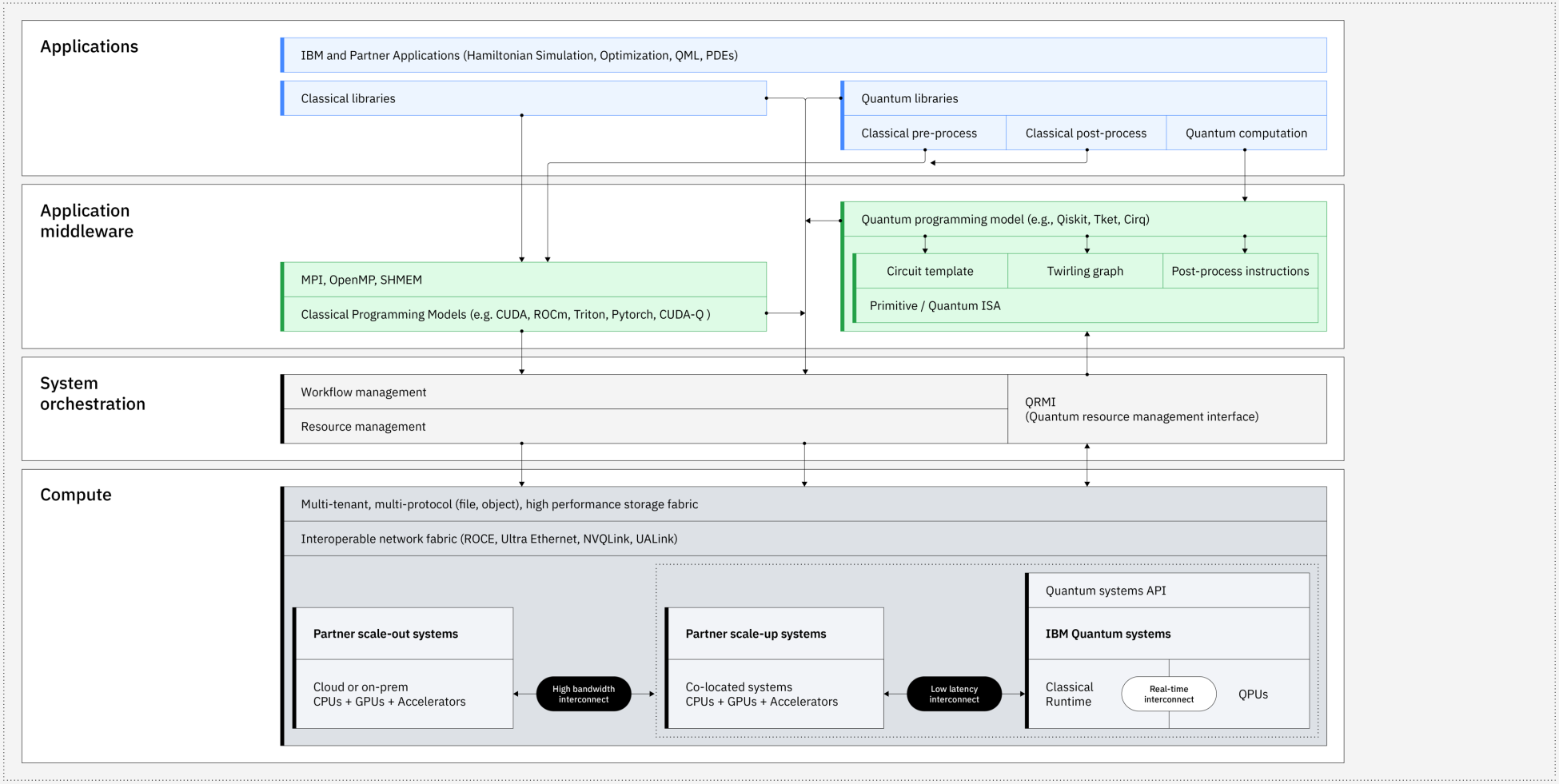
- Coupled scheduling across Quantum and Classical resources
  - Historically schedulers optimize for dominant fair resource scheduling (CPUs, GPUs, Nodes)

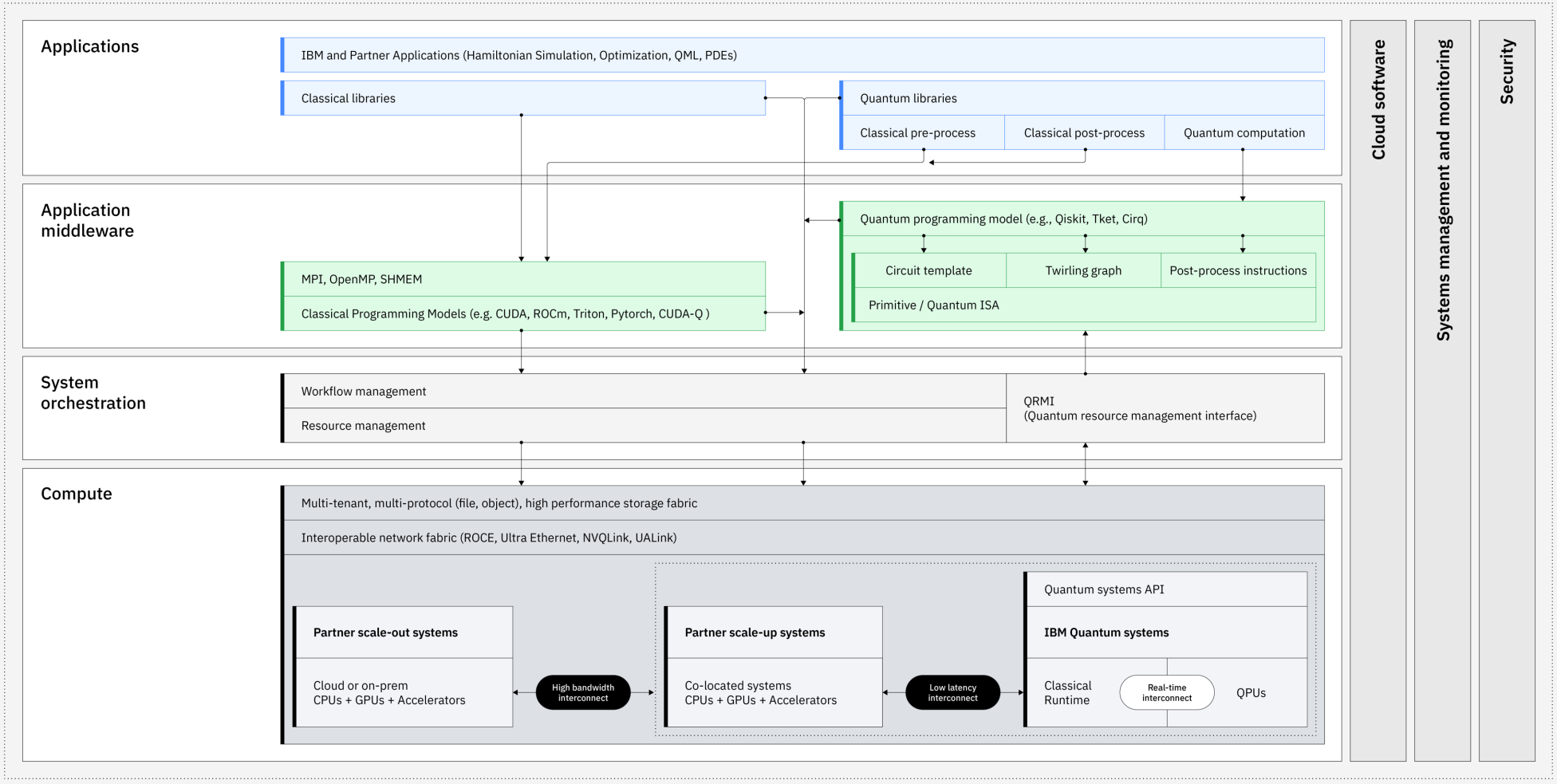
- In the short run we need to address:
  - Keep HPC systems utilized for HPC only jobs
  - Keep Quantum Systems for QPU only jobs
  - Support timely execution of coupled HPC+QPU jobs

Quantum Resource Management Interface (QRMI)

- Thin and vendor agnostic layer to access, control and monitor underlying on-prem or cloud Quantum systems
- Exposes notion of quantum resources, which might be qubit, entire physical system, virtual QPU, etc.
- 4 sets of APIs available for QRMI:
  - Resource acquisition: availability, acquire, release.
  - Task running: execution on quantum payload on target hardware
  - [2026 1H] Beta of Accounting, Resource discovery etc.
- Provides loose coupling between clients and systems




Applications
IBM and Partner Applications (Hamiltonian Simulation, Optimization, QML, PDEs)
Classical libraries
Quantum libraries
Classical pre-process
Classical post-process
Quantum computation
Application middleware
Quantum programming model (e.g., Qiskit, Tket, Cirq)
MPI, OpenMP, SHMEM
Classical Programming Models (e.g. CUDA, ROCm, Triton, Pytorch, CUDA-Q)
Circuit template
Twirling graph
Post-process instructions
Primitive / Quantum ISA
System orchestration
Workflow management
Resource management
QRMI
(Quantum resource management interface)
Compute
Multi-tenant, multi-protocol (file, object), high performance storage fabric
Interoperable network fabric (ROCE, Ultra Ethernet, NVQLink, UALink)
Partner scale-out systems
Cloud or on-prem
CPUs + GPUs + Accelerators
High bandwidth interconnect
Partner scale-up systems
Co-located systems
CPUs + GPUs + Accelerators
Low latency interconnect
Quantum systems API
IBM Quantum systems
Classical Runtime
Real-time interconnect
QPUs


Applications
IBM and Partner Applications (Hamiltonian Simulation, Optimization, QML, PDEs)
Classical libraries
Quantum libraries
Classical pre-process
Classical post-process
Quantum computation
Application middleware
Quantum programming model (e.g., Qiskit, Tket, Cirq)
MPI, OpenMP, SHMEM
Classical Programming Models (e.g. CUDA, ROCm, Triton, Pytorch, CUDA-Q)
Circuit template
Twirling graph
Post-process instructions
Primitive / Quantum ISA
System orchestration
Workflow management
Resource management
QRMI
(Quantum resource management interface)
Compute
Multi-tenant, multi-protocol (file, object), high performance storage fabric
Interoperable network fabric (ROCE, Ultra Ethernet, NVQLink, UALink)
Partner scale-out systems
Cloud or on-prem
CPUs + GPUs + Accelerators
High bandwidth interconnect
Partner scale-up systems
Co-located systems
CPUs + GPUs + Accelerators
Low latency interconnect
Quantum systems API
IBM Quantum systems
Classical Runtime
Real-time interconnect
QPUs
Cloud software
Systems management and monitoring
Security

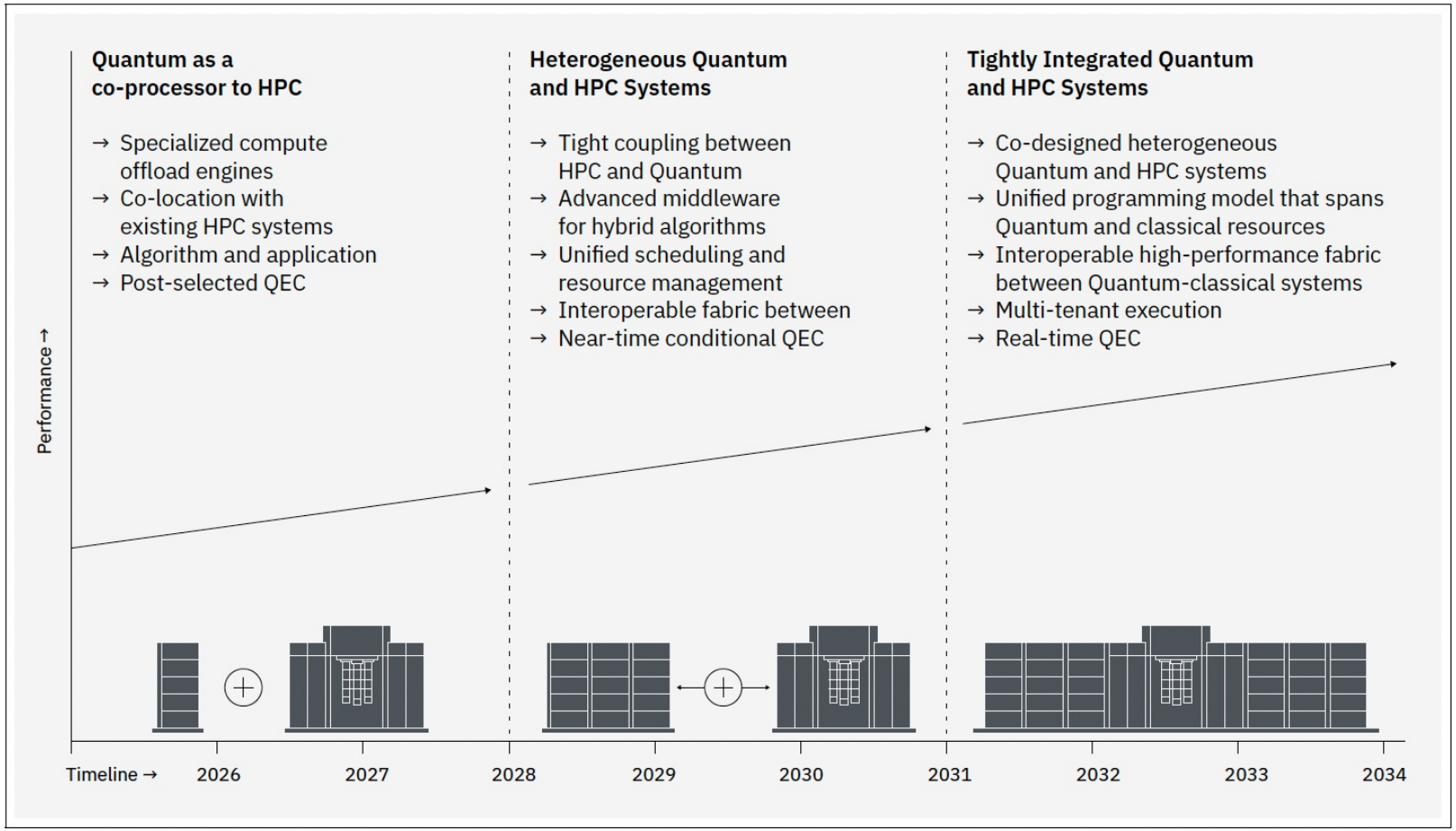
Outline

5 use cases for Quantum + HPC

1 Reference Architecture

3 Phased Deployment

QCSC: An integration roadmap



Summary

- Vision: Quantum + classical systems to tackle complex problems
- Architecture: Staged integration guided by key use cases
- Stack: Layered system for hybrid quantum–classical workloads
- Roadmap: Phased evolution with open collaboration